

CSE 220 DFT–DTFT–FFT Online Lab Survival Guide

Theory → Assignment Code → Online Problem Solving

Prepared from the supplied course slides, current assignment, and three previous online questions

Important

This guide is intentionally larger than a normal note. It is designed for two jobs: (1) study before the online lab test, and (2) serve as a searchable code-and-formula reference during the test. The “prediction” sections are informed guesses from the supplied previous questions and the current assignment; they are not guarantees of what the instructor will ask.

Contents

1	What the online test is really testing	4
2	Source map: what each supplied resource contributes	4
3	Core theory you must be able to translate into code	5
3.1	DFT and IDFT	5
3.2	DFT versus FFT	5
3.3	DTFT and the DFT sampling viewpoint	5
3.4	The clock model: all DFT indexing wraps	6
4	The reusable assignment toolbox	6
4.1	DFTAnalyzer: direct definition	6
4.2	FFTTransformer: iterative radix-2 DIT	6
4.3	ArbitraryLengthFFT: when length is not a power of two	7
4.4	One universal verification pattern	7
5	Previous Online A: huge integer multiplication	8
5.1	What the old question asked	8
5.2	Why integer multiplication becomes convolution	8
5.3	Why the padding rule matters	8
5.4	Carry propagation	8
5.5	Complete self-contained solution file	9
6	Previous Online B: weighted polynomial multiplication	11
6.1	Recognize the hidden convolution	11
6.2	The output-order inconsistency in the supplied old handout	11
6.3	Example 1	11
6.4	Complete self-contained solution file	11
7	Previous Online C: row-wise image shift correction	14
7.1	What the old question asked	14
7.2	Cross-correlation in the frequency domain	14
7.3	Why correlation is related to the properties lecture	14
7.4	RGB handling	14
7.5	Power-of-two warning	14

7.6	JPEG verification warning	14
7.7	Complete self-contained solution file	15
8	The property-first mindset	19
9	DFT properties cookbook: formula, logic, and lab-ready code	20
9.1	Linearity	20
9.2	DFT periodicity	20
9.3	Circular time shift	20
9.4	Frequency shift / modulation	21
9.5	Circular time reversal	22
9.6	Conjugation	22
9.7	Conjugate symmetry for real signals	22
9.8	Parseval's relation	23
9.9	Duality	23
9.10	Circular convolution theorem	23
9.11	Linear convolution via DFT/FFT	24
9.12	Multiplication property	25
9.13	Circular correlation and shift detection	25
10	2-D transforms and image questions	26
10.1	2-D transform template	26
10.2	2-D linear convolution shape	26
10.3	Color images	26
10.4	Circular image convolution and kernel centering	27
11	How to predict the new online from the old online and current assignment	28
11.1	Likelihood tiers	28
11.2	A fast decision tree during the exam	28
12	Twenty predicted practice questions with solutions	30
12.1	Practice 1 – DFT/IDFT round trip and the missing $1/N$ bug	30
12.2	Practice 2 – Verify linearity in two independent ways	31
12.3	Practice 3 – Circular time shift: compute spectrum without transforming the shifted signal	32
12.4	Practice 4 – Modulation shifts the spectrum	33
12.5	Practice 5 – Time reversal and conjugation	34
12.6	Practice 6 – Reconstruct the missing half of the DFT of a real signal	35
12.7	Practice 7 – Parseval: compare energy in time and frequency	36
12.8	Practice 8 – Verify duality	37
12.9	Practice 9 – Circular convolution: direct loops versus FFT	38
12.10	Practice 10 – Demonstrate that zero-padding turns circular convolution into linear convolution	39
12.11	Practice 11 – Verify the multiplication property	40
12.12	Practice 12 – Big-integer multiplication, but with a different base	41
12.13	Practice 13 – Weighted polynomial convolution with a changed weighting rule	42
12.14	Practice 14 – Detect a circular shift of one 1-D signal	43
12.15	Practice 15 – Correct row-wise image distortion	44
12.16	Practice 16 – Blur a small image with a kernel using your assignment	45
12.17	Practice 17 – Build a simple frequency-domain filter	46
12.18	Practice 18 – Zero-padding as finer samples of the same DTFT	47
12.19	Practice 19 – Compare naive DFT and FFT results and complexity	48
12.20	Practice 20 – FFT implementation / debugging question	49
13	DTFT essentials that may still appear in an online	50
13.1	Evaluate the DTFT numerically on a dense grid	50
13.2	Verify 2π periodicity of the DTFT	50
13.3	Verify that the DFT samples the DTFT	50

13.4 Map DFT bins to physical frequency	51
13.5 Zero-padding and old-bin consistency	51
14 How to reuse the assignment OOP structure in a new online file	52
14.1 Pattern 1 – property function with an engine parameter	52
14.2 Pattern 2 – choose engine by a string	52
14.3 Pattern 3 – add a new class without changing the application	52
14.4 Pattern 4 – modify an existing assignment function minimally	53
15 Lab-ready copy blocks	54
15.1 Block A – standard imports	54
15.2 Block B – safe linear convolution	54
15.3 Block C – same-length circular convolution	54
15.4 Block D – circular correlation and shift	54
15.5 Block E – property verification	55
15.6 Block F – circular reversal indices	55
15.7 Block G – right circular shift and its phase	55
15.8 Block H – modulation	55
15.9 Block I – Parseval	55
15.10Block J – real-signal conjugate symmetry	55
15.11Block K – integer cleanup after IFFT	56
16 Python and NumPy syntax cheat sheet for this online	57
16.1 Array creation and type	57
16.2 Zeros and padding	57
16.3 Complex numbers	57
16.4 Complex exponentials	57
16.5 Useful indexing	57
16.6 Comparing numerical results	57
16.7 Finding a correlation peak	57
16.8 Rounding values known to be integers	57
16.9 2-D / image indexing	57
16.10Power-of-two test	58
16.11Bit shifts used in the FFT code	58
17 Common failure modes and how to diagnose them	59
18 Complexity cheat sheet	60
19 What to memorize versus what to search in this PDF	60
20 Extra mini-prompts for fast drill	61
21 Final formula sheet	62
22 30-minute online workflow	64
23 Source-grounded closing summary	64

1 What the online test is really testing

The supplied previous online questions and the current assignment all point to the same pattern: you are rarely asked to “draw a spectrum” just for its own sake. Instead, the DFT is used as a computational tool. The online usually gives a concrete operation in the time/spatial domain and expects you to recognize a DFT property that makes the operation easier.

The high-value recognition patterns are:

If the question says ...	Think ...
“shift”, “delay”, “rotate”, “circular shift”	circular time-shift property; phase multiplication in the DFT
“find unknown shift”, “align”, “registration”	circular cross-correlation via FFT/IFFT
“convolution”, “filter”, “blur”, “polynomial product”	transform, multiply spectra, inverse transform
“linear convolution”	zero-pad to at least $L + M - 1$ before FFT
“multiply two huge integers”	digits/limbs are polynomial coefficients; multiplication is convolution
“verify a property in two ways”	compute left side directly and right side from the property, then <code>np.allclose</code>
“real signal”	conjugate symmetry $X[N - k] = X^*[k]$
“energy”	Parseval
“modulate”, “multiply by complex tone”	frequency shift
“reverse”	time reversal maps to reversed DFT bins

Exam Tip

Before coding, write one sentence: “*This is a [shift / convolution / correlation / modulation / symmetry] problem, so I will use [property].*” That one sentence usually determines the entire solution.

2 Source map: what each supplied resource contributes

This guide is based on the following supplied material.

- **Lecture 1 – DTFT and DFT.** Core DFT/IDFT definition, DTFT, DFT as samples of the DTFT, bin interpretation, and zero-padding as finer sampling of the same DTFT. The DFT/IDFT pair is on slide page 17; the DFT-as-sampled-DTFT view is on page 29; zero-padding is on page 31.
- **Lecture 2 – DFT Properties and Applications.** This is the most directly useful lab-test lecture: linearity, circular shifts, modulation, reversal, conjugation, real-signal symmetry, duality, convolution, multiplication, Parseval, and the zero-padding rule that converts circular convolution into linear convolution. The compact property table is on page 12; the zero-padding rule is on page 22; the filtering recipe is on page 23.
- **Fast Fourier Transform.** Radix-2 DIT/DIF derivation, butterflies, bit-reversed ordering, $O(N \log N)$ complexity, and the power-of-two requirement. The current assignment implements iterative radix-2 DIT.
- **Current assignment repository.** The shared `transforms.py` contains `DFTAnalyzer`, `FFTTransformer`, `ArbitraryLengthFFT`, and `next_power_of_two`. `bigmul.py` uses transform-domain convolution for large integers. `image_conv.py` performs separable 2-D transforms and image convolution.
- **Previous Online Section A.** Big non-negative integer multiplication from LSD-first digit arrays using FFT/IFFT circular convolution plus carry propagation.

- **Previous Online Section B.** Weighted polynomial multiplication using FFT/IFFT convolution.
- **Previous Online Section C.** Detect an independent circular horizontal shift for every image row by DFT-based cross-correlation, then undo the shifts.

3 Core theory you must be able to translate into code

3.1 DFT and IDFT

For a length- N sequence:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1.$$

The inverse is:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+j2\pi kn/N}.$$

The sign and scale rule is worth memorizing:

Operation	Exponent	Scale
DFT / FFT	$-j$	no $1/N$
IDFT / IFFT	$+j$	divide by N

3.2 DFT versus FFT

The FFT is not a different transform. It computes the same $X[k]$ as the DFT definition.

$$\text{naive DFT: } O(N^2), \quad \text{radix-2 FFT: } O(N \log_2 N).$$

The current assignment uses radix-2 decimation-in-time. Its essential butterfly is

$$\begin{aligned} u &= G[k], & v &= W_m^k H[k], \\ \text{top} &= u + v, & \text{bottom} &= u - v. \end{aligned}$$

For radix-2, the transform length must be a power of two. That is why the assignment repeatedly uses `next_power_of_two`.

3.3 DTFT and the DFT sampling viewpoint

For a finite sequence:

$$X(e^{j\omega}) = \sum_n x[n] e^{-j\omega n}.$$

The DTFT is continuous in ω and 2π -periodic.

The N -point DFT is the DTFT sampled at

$$\omega_k = \frac{2\pi k}{N}.$$

Thus:

$$X[k] = X(e^{j\omega}) \Big|_{\omega=2\pi k/N}.$$

This is important in online questions involving frequency bins, zero-padding, or “compare the DTFT and the DFT”.

3.4 The clock model: all DFT indexing wraps

The properties slide uses the most useful mental model: on N samples, indices live on a clock.

$$((n))_N = n \bmod N.$$

Therefore:

$$x[-1] \text{ in circular indexing means } x[N-1].$$

In Python, this is exactly what `np.roll` and modular indexing do.

```
N = len(x)
idx = (-np.arange(N)) % N      # 0, N-1, N-2, ...
x_reversed_circular = x[idx]

x_shifted_right_by_m = np.roll(x, m)
```

4 The reusable assignment toolbox

4.1 DFTAnalyzer: direct definition

The assignment's direct DFT constructs the matrix

$$W[k, n] = e^{-j2\pi kn/N}$$

and returns Wx . The inverse uses the positive sign and divides by N .

A compact standalone equivalent is:

```
def dft_direct(x):
    x = np.asarray(x, dtype=np.complex128).reshape(-1)
    N = len(x)
    n = np.arange(N)
    W = np.exp(-2j*np.pi*n[:,None]*n[None,:]/N)
    return W @ x

def idft_direct(X):
    X = np.asarray(X, dtype=np.complex128).reshape(-1)
    N = len(X)
    k = np.arange(N)
    W = np.exp(+2j*np.pi*k[:,None]*k[None,:]/N)
    return (W @ X) / N
```

Use the direct DFT when the online explicitly asks you to verify the definition, show $O(N^2)$ behavior, or work with a small non-power-of-two N without the arbitrary-length FFT.

4.2 FFTTransformer: iterative radix-2 DIT

The assignment performs:

1. bit-reversal permutation;
2. stages with sizes $m = 2, 4, 8, \dots, N$;
3. $N/2$ butterflies per stage;
4. divide by N only for the inverse.

The exact assignment pattern that you should recognize is:

```
m = 2
sign = 1.0 if inverse else -1.0
while m <= N:
    half = m // 2
    tw = np.exp(sign * 2j*np.pi*np.arange(half)/m)
```

```

for start in range(0, N, m):
    u = a[start:start+half].copy()
    v = tw * a[start+half:start+m]
    a[start:start+half] = u + v
    a[start+half:start+m] = u - v

m <<= 1

if inverse:
    a /= N

```

4.3 ArbitraryLengthFFT: when length is not a power of two

The bonus `ArbitraryLengthFFT` uses Bluestein's algorithm. Its practical value in an online is simple: if a genuinely circular problem has length N that is not a power of two, you cannot zero-pad without changing the period of the circular problem. In that case, use the arbitrary-length engine.

Examples:

- linear convolution: padding is fine and normally required;
- circular correlation of an image row whose true width is 913: do *not* change the period to 1024 just to use radix-2; use `ArbitraryLengthFFT`.

4.4 One universal verification pattern

```

left = ...           # compute directly
right = ...          # compute using DFT property

print("max error =", np.max(np.abs(left - right)))
print("match =", np.allclose(left, right, atol=1e-9))

```

Do not compare floating-point Fourier results with `==`. Tiny errors such as 10^{-13} are normal.

5 Previous Online A: huge integer multiplication

5.1 What the old question asked

The input consists of two non-negative integers stored as arrays of base-10 digits from least-significant digit to most-significant digit. For example:

$$123 \leftrightarrow [3, 2, 1].$$

The required output is also LSD-first.

The old online explicitly required FFT/IFFT-based circular convolution, with zero-padding to at least $m + n - 1$, followed by decimal carry propagation.

5.2 Why integer multiplication becomes convolution

Write:

$$A = \sum_i a_i 10^i, \quad B = \sum_j b_j 10^j.$$

Then:

$$AB = \sum_k \left(\sum_{i+j=k} a_i b_j \right) 10^k.$$

The raw coefficient

$$C[k] = \sum_{i+j=k} a_i b_j$$

is exactly the linear convolution of the digit arrays.

So the algorithm is:

$$A \xrightarrow{\text{FFT}} A_f, \quad B \xrightarrow{\text{FFT}} B_f, \quad C = \text{IFFT}(A_f B_f),$$

then round the real part and propagate carries.

5.3 Why the padding rule matters

If the digit lengths are m and n , the linear convolution length is

$$m + n - 1.$$

Without sufficient zero-padding, multiplying DFTs gives circular convolution and the high-order coefficients wrap into the low-order coefficients. That destroys the integer product.

For radix-2:

$$N = \text{next power of two at least } m + n - 1.$$

5.4 Carry propagation

The raw convolution for 123×45 is

$$[15, 22, 13, 4].$$

Normalize in base 10:

$$15 \rightarrow 5 \text{ carry } 1,$$

$$22 + 1 = 23 \rightarrow 3 \text{ carry } 2,$$

$$13 + 2 = 15 \rightarrow 5 \text{ carry } 1,$$

$$4 + 1 = 5.$$

Result:

$$[5, 3, 5, 5] \leftrightarrow 5535.$$

Exam Tip

This previous question is extremely close to the current assignment's `bigmul.py`. The only major representation change is that the current assignment groups four decimal digits per limb, whereas the old online used one decimal digit per coefficient.

5.5 Complete self-contained solution file

Listing 1: Previous Online A – complete solution

```

"""Previous-year Online Section A: big-integer multiplication from LSD-first digits.

FFT/IFFT-based circular convolution is zero-padded so it equals the required
linear convolution. The FFT core is copied from our assignment so this is
self-contained and can be submitted as one Python file.
"""

import numpy as np

def next_power_of_two(n):
    """Return the smallest power of two >= n, with minimum 1."""
    if n < 1:
        return 1
    p = 1
    while p < n:
        p <<= 1
    return p

class FFTTransformer:
    """Radix-2 decimation-in-time Cooley-Tukey FFT copied from our assignment."""

    name = "fft"

    @staticmethod
    def _core(x, inverse=False):
        a = np.asarray(x, dtype=np.complex128).reshape(-1).copy()
        N = a.size
        if N == 0:
            return a
        if N & (N - 1):
            raise ValueError("radix-2 FFT requires a power-of-two length")

        j = 0
        for i in range(1, N):
            bit = N >> 1
            while j & bit:
                j ^= bit
                bit >>= 1
            j ^= bit
            if i < j:
                a[i], a[j] = a[j], a[i]

        m = 2
        sign = 1.0 if inverse else -1.0
        while m <= N:
            half = m // 2
            tw = np.exp(sign * 2j * np.pi * np.arange(half) / m)
            for start in range(0, N, m):
                u = a[start:start + half].copy()
                v = tw * a[start + half:start + m]
                a[start:start + half] = u + v
                a[start + half:start + m] = u - v
            m <<= 1

```

```

        if inverse:
            a /= N
        return a

    def transform(self, x):
        return self._core(x, inverse=False)

    def inverse(self, spectrum):
        return self._core(spectrum, inverse=True)

def multiply_digit_arrays(A, B):
    A = np.asarray(A, dtype=np.int64).reshape(-1)
    B = np.asarray(B, dtype=np.int64).reshape(-1)

    if A.size == 0 or B.size == 0:
        return [0]
    if np.any((A < 0) | (A > 9)) or np.any((B < 0) | (B > 9)):
        raise ValueError("A and B must contain digits 0..9")

    needed = A.size + B.size - 1
    N = next_power_of_two(needed)

    ap = np.zeros(N, dtype=np.complex128)
    bp = np.zeros(N, dtype=np.complex128)
    ap[:A.size] = A
    bp[:B.size] = B

    fft = FFTTransformer()
    raw = fft.inverse(fft.transform(ap) * fft.transform(bp))
    coeff = np rint(raw.real[:needed]).astype(np.int64).tolist()

    carry = 0
    i = 0
    while i < len(coeff) or carry:
        if i == len(coeff):
            coeff.append(0)
        total = coeff[i] + carry
        carry, coeff[i] = divmod(total, 10)
        i += 1

    while len(coeff) > 1 and coeff[-1] == 0:
        coeff.pop()
    return coeff

def digits_to_int(digits):
    return sum(int(d) * 10**i for i, d in enumerate(digits))

if __name__ == "__main__":
    ans1 = multiply_digit_arrays([3, 2, 1], [5, 4])
    print("Example 1:", ans1)
    assert ans1 == [5, 3, 5, 5]

    ans2 = multiply_digit_arrays([9, 9, 9], [9, 9])
    print("Example 2:", ans2)
    assert ans2 == [1, 0, 9, 8, 9]

```

6 Previous Online B: weighted polynomial multiplication

6.1 Recognize the hidden convolution

If

$$P(x) = p_m x^m + \cdots + p_0, \quad Q(x) = q_n x^n + \cdots + q_0$$

and each coefficient of P is multiplied by a corresponding weight, define a weighted coefficient sequence

$$\tilde{p}_i = w_i p_i.$$

Then the requested weighted product is simply the polynomial product

$$R(x) = \tilde{P}(x)Q(x),$$

so its coefficients are a convolution.

This is the same FFT pipeline as big-integer multiplication, but there is no carry propagation.

6.2 The output-order inconsistency in the supplied old handout

There is an important inconsistency in the supplied previous-year Section B sheet:

- Example 1 is printed in descending-power order.
- The listed Example 2 result is the reverse orientation of the descending-power convolution.

The mathematical weighted product itself is unambiguous; only the printed list orientation conflicts. Therefore the solution below makes the output order explicit rather than silently guessing. In a real online, follow the exact order stated by that question and its examples.

6.3 Example 1

Given descending arrays

$$P = [1, 3, 2], \quad Q = [4, 1], \quad W = [3, 2, 1],$$

first weight P elementwise:

$$P \odot W = [3, 6, 2].$$

Then convolve:

$$[3, 6, 2] * [4, 1] = [12, 27, 14, 2].$$

6.4 Complete self-contained solution file

Listing 2: Previous Online B – complete solution

```
"""Previous-year Online Section B: weighted polynomial product by FFT/IFFT.

The handout is internally inconsistent about OUTPUT ORDER:
- Example 1 is printed in descending-power order.
- Example 2's listed result is the reverse (ascending-power order).

Therefore this solution exposes an explicit output_order argument rather than
silently guessing. The underlying weighted product is unambiguous.
"""

import numpy as np

def next_power_of_two(n):
    """Return the smallest power of two >= n, with minimum 1."""
    if n < 1:
        return 1
    p = 1
    while p < n:
```

```

        p <= 1
    return p

class FFTTransformer:
    """Radix-2 decimation-in-time Cooley-Tukey FFT copied from our assignment."""

    name = "fft"

    @staticmethod
    def _core(x, inverse=False):
        a = np.asarray(x, dtype=np.complex128).reshape(-1).copy()
        N = a.size
        if N == 0:
            return a
        if N & (N - 1):
            raise ValueError("radix-2 FFT requires a power-of-two length")

        j = 0
        for i in range(1, N):
            bit = N >> 1
            while j & bit:
                j ^= bit
                bit >>= 1
            j ^= bit
            if i < j:
                a[i], a[j] = a[j], a[i]

        m = 2
        sign = 1.0 if inverse else -1.0
        while m <= N:
            half = m // 2
            tw = np.exp(sign * 2j * np.pi * np.arange(half) / m)
            for start in range(0, N, m):
                u = a[start:start + half].copy()
                v = tw * a[start + half:start + m]
                a[start:start + half] = u + v
                a[start + half:start + m] = u - v
            m <= 1

        if inverse:
            a /= N
        return a

    def transform(self, x):
        return self._core(x, inverse=False)

    def inverse(self, spectrum):
        return self._core(spectrum, inverse=True)

def weighted_polynomial_product(P_desc, Q_desc, W_desc, output_order="descending"):
    """Compute the weighted product.

    P_desc, Q_desc, W_desc are aligned in descending powers.
    output_order:
        "descending" -> polynomial-style coefficients, highest power first.
        "ascending" -> constant term first.
    """
    P = np.asarray(P_desc, dtype=np.int64).reshape(-1)
    Q = np.asarray(Q_desc, dtype=np.int64).reshape(-1)
    W = np.asarray(W_desc, dtype=np.int64).reshape(-1)

    if P.size == 0 or Q.size == 0:
        return []
    if W.size != P.size:
        raise ValueError("W must contain one weight for every coefficient of P")

```

```

weighted_P_desc = P * W
needed = weighted_P_desc.size + Q.size - 1
N = next_power_of_two(needed)

ap = np.zeros(N, dtype=np.complex128)
bp = np.zeros(N, dtype=np.complex128)
ap[:weighted_P_desc.size] = weighted_P_desc
bp[:Q.size] = Q

fft = FFTTransformer()
conv_desc = np.rint(
    fft.inverse(fft.transform(ap) * fft.transform(bp)).real[:needed]
).astype(np.int64).tolist()

if output_order == "descending":
    return conv_desc
if output_order == "ascending":
    return conv_desc[::-1]
raise ValueError("output_order must be 'descending' or 'ascending'")

if __name__ == "__main__":
    # Matches Example 1 exactly as printed.
    ex1 = weighted_polynomial_product([1, 3, 2], [4, 1], [3, 2, 1], "descending")
    print("Example 1 (descending):", ex1)
    assert ex1 == [12, 27, 14, 2]

    # Example 2 in the handout is printed in the opposite orientation.
    ex2 = weighted_polynomial_product(
        [1, 3, 2, 6, 7], [4, 1], [3, 2, 1, 5, 6], "ascending"
    )
    print("Example 2 (ascending, as printed in handout):", ex2)
    assert ex2 == [42, 198, 122, 14, 27, 12]

    print("If your actual online states one output order clearly, use that order.")

```

7 Previous Online C: row-wise image shift correction

7.1 What the old question asked

The distorted image was created by circularly shifting each row horizontally by a different amount. No vertical shift occurred. The required method was DFT-based cross-correlation using FFT/IFFT.

This is a property question disguised as an image problem.

7.2 Cross-correlation in the frequency domain

For two same-length signals $x[n]$ and $y[n]$, a convenient circular cross-correlation convention is:

$$r_{xy}[m] = \text{IDFT} \{X^*[k]Y[k]\}.$$

If

$$y[n] = x[(n - s)_N]$$

which is a circular right shift of x by s , then the correlation peaks at $m = s$ under this convention.

Therefore the row algorithm is:

1. take one row from the original image;
2. take the corresponding shifted row;
3. compute FFTs;
4. multiply $\overline{X[k]}Y[k]$;
5. IFFT;
6. `argmax` gives the shift;
7. undo it with `np.roll(shifted_row, -shift)`.

7.3 Why correlation is related to the properties lecture

Cross-correlation is not printed as its own row in the supplied property table, but it follows from the same ingredients: conjugation, reversal, and circular convolution. That is exactly why it can be computed by one product of spectra.

7.4 RGB handling

The horizontal shift is the same for all color channels in a row. A robust implementation computes a correlation for each channel and adds them. Subtracting the channel mean removes the large DC component so the structure of the row dominates the peak.

7.5 Power-of-two warning

This is a *circular* problem with period equal to the true image width. If width W is not a power of two, padding to a new width changes the circular period. The current assignment's `ArbitraryLengthFFT` is therefore a clean solution.

7.6 JPEG verification warning

The supplied old statement names the distorted file as JPEG while asking for a perfect pixel match. JPEG is normally lossy, so exact pixel equality can be impossible even after every geometric shift has been corrected. The code reports both geometric correction statistics and exact equality.

7.7 Complete self-contained solution file

Listing 3: Previous Online C – complete solution

```

"""Previous-year Online Section C: undo row-wise circular image shifts.

For each row:
    r = IFFT(conj(FFT(original_row)) * FFT(shifted_row))
The index of the largest correlation peak is the circular RIGHT shift.
Undo it with np.roll(shifted_row, -shift).

Includes our assignment's radix-2 FFT and arbitrary-length Bluestein FFT.
"""

import argparse
import numpy as np
from PIL import Image

def next_power_of_two(n):
    if n < 1:
        return 1
    p = 1
    while p < n:
        p <<= 1
    return p

class FFTTransformer:
    name = "fft"

    @staticmethod
    def _core(x, inverse=False):
        a = np.asarray(x, dtype=np.complex128).reshape(-1).copy()
        N = a.size
        if N == 0:
            return a
        if N & (N - 1):
            raise ValueError("radix-2 FFT requires a power-of-two length")

        j = 0
        for i in range(1, N):
            bit = N >> 1
            while j & bit:
                j ^= bit
                bit >>= 1
            j ^= bit
            if i < j:
                a[i], a[j] = a[j], a[i]

        m = 2
        sign = 1.0 if inverse else -1.0
        while m <= N:
            half = m // 2
            tw = np.exp(sign * 2j * np.pi * np.arange(half) / m)
            for start in range(0, N, m):
                u = a[start:start + half].copy()
                v = tw * a[start + half:start + m]
                a[start:start + half] = u + v
                a[start + half:start + m] = u - v
            m <<= 1

        if inverse:
            a /= N
        return a

    def transform(self, x):
        return self._core(x, inverse=False)

```

```

def inverse(self, spectrum):
    return self._core(spectrum, inverse=True)

class ArbitraryLengthFFT(FFTTransformer):
    name = "arbitrary"

    def transform(self, x):
        x = np.asarray(x, dtype=np.complex128).reshape(-1)
        N = x.size
        if N == 0:
            return x.copy()
        if (N & (N - 1)) == 0:
            return super().transform(x)

        M = next_power_of_two(2 * N - 1)
        n = np.arange(N, dtype=np.int64)
        n2 = n.astype(np.float64) ** 2
        chirp = np.exp(-1j * np.pi * n2 / N)

        a = np.zeros(M, dtype=np.complex128)
        a[:N] = x * chirp

        b = np.zeros(M, dtype=np.complex128)
        b[:N] = np.exp(1j * np.pi * n2 / N)
        if N > 1:
            b[M - N + 1:] = b[1:N][::-1]

        A = super().transform(a)
        B = super().transform(b)
        conv = super().inverse(A * B)
        return chirp * conv[:N]

    def inverse(self, spectrum):
        X = np.asarray(spectrum, dtype=np.complex128).reshape(-1)
        N = X.size
        if N == 0:
            return X.copy()
        return np.conjugate(self.transform(np.conjugate(X))) / N

def _as_channels(row):
    row = np.asarray(row, dtype=np.float64)
    return row[:, None] if row.ndim == 1 else row

def detect_right_shift(original_row, shifted_row, engine=None):
    """Return s such that shifted_row ~= np.roll(original_row, s, axis=0)."""
    if engine is None:
        engine = ArbitraryLengthFFT()

    a = _as_channels(original_row)
    b = _as_channels(shifted_row)
    if a.shape != b.shape:
        raise ValueError("row shapes do not match")

    width, channels = a.shape
    corr = np.zeros(width, dtype=np.float64)
    usable = 0

    for c in range(channels):
        x = a[:, c] - np.mean(a[:, c])
        y = b[:, c] - np.mean(b[:, c])
        if np.linalg.norm(x) < 1e-12 or np.linalg.norm(y) < 1e-12:
            continue

```

```

        X = engine.transform(x)
        Y = engine.transform(y)
        corr += engine.inverse(np.conjugate(X) * Y).real
        usable += 1

    if usable == 0:
        return 0
    return int(np.argmax(corr))

def correct_rolling_shutter(original, shifted):
    original = np.asarray(original, dtype=np.float64)
    shifted = np.asarray(shifted, dtype=np.float64)

    if original.shape != shifted.shape:
        raise ValueError("original and shifted images must have the same shape")
    if original.ndim not in (2, 3):
        raise ValueError("expected HxW or HxWxC image")

    H = original.shape[0]
    corrected = shifted.copy()
    shifts = np.zeros(H, dtype=np.int64)
    engine = ArbitraryLengthFFT()

    for r in range(H):
        s = detect_right_shift(original[r], shifted[r], engine)
        shifts[r] = s
        corrected[r] = np.roll(shifted[r], -s, axis=0)

    return corrected, shifts

def load_rgb(path):
    return np.asarray(Image.open(path).convert("RGB"), dtype=np.float64)

def save_rgb(array, path):
    out = np.clip(np rint(array), 0, 255).astype(np.uint8)
    Image.fromarray(out, mode="RGB").save(path)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("original")
    ap.add_argument("shifted")
    ap.add_argument("output", nargs="?", default="corrected.png")
    args = ap.parse_args()

    original = load_rgb(args.original)
    shifted = load_rgb(args.shifted)
    corrected, shifts = correct_rolling_shutter(original, shifted)
    save_rgb(corrected, args.output)

    err = np.abs(corrected - original)
    print("saved:", args.output)
    print("detected shifts:", shifts.tolist())
    print("shift range:", int(shifts.min()), "to", int(shifts.max()))
    print("max absolute pixel error:", float(err.max()))
    print("mean absolute pixel error:", float(err.mean()))
    print("exact pixel match:",
          bool(np.array_equal(np rint(corrected).astype(np.uint8),
                              np rint(original).astype(np.uint8))))
    print("JPEG note: lossy compression can prevent exact pixel equality even")
    print("when the geometric row shifts are detected and undone correctly.")

if __name__ == "__main__":

```

```
main()
```

8 The property-first mindset

The most likely way to lose time in the online is to start writing loops before deciding which DFT property applies. For every property question, use the same four-step method:

1. **Name the property.**
2. **Write the formula.**
3. **Compute the left side directly.**
4. **Compute the right side using the original DFT and compare.**

The next section is a code cookbook for exactly that workflow.

9 DFT properties cookbook: formula, logic, and lab-ready code

Throughout this section, assume both sequences have the same length N unless the property explicitly says otherwise. For radix-2 FFT examples, choose N as a power of two.

A useful common setup is:

Listing 4: Common setup

```
import numpy as np
from transforms import DFTAnalyzer, FFTTransformer, ArbitraryLengthFFT

engine = FFTTransformer()      # choose N = 8, 16, 32, ...
x = np.array([3, 5, 2, 0, 1, 2, 4, 1], dtype=float)
N = len(x)
X = engine.transform(x)
```

9.1 Linearity

$$ax[n] + by[n] \longleftrightarrow aX[k] + bY[k].$$

Logic. The DFT is a sum, and sums distribute over addition and scalar multiplication.

Typical online wording. “Verify the linearity property for two supplied sequences and constants.”

Listing 5: Verify DFT linearity

```
a = 2.5
b = -1.25

x = np.array([1,2,3,4,0,0,0,0], dtype=float)
y = np.array([4,1,0,2,3,0,0,0], dtype=float)

X = engine.transform(x)
Y = engine.transform(y)

left = engine.transform(a*x + b*y)
right = a*X + b*Y

print(np.allclose(left, right, atol=1e-9))
print("max error =", np.max(np.abs(left-right)))
```

9.2 DFT periodicity

$$X[k + N] = X[k].$$

A stored DFT array only contains $k = 0, \dots, N - 1$, so periodicity is represented by modular indexing:

$$X[k] = X[k \bmod N].$$

Listing 6: Frequency-bin periodic indexing

```
k = 13
same_bin = X[k % N]
print("X[13] means X[13 mod N] =", same_bin)
```

9.3 Circular time shift

The slide convention is:

$$x[((n - m))_N] \longleftrightarrow W_N^{km} X[k] = e^{-j2\pi km/N} X[k].$$

A positive m is a circular shift to the right:

$$y[n] = x[((n - m))_N].$$

In NumPy:

$$y = \text{np.roll}(x, m).$$

Listing 7: Verify circular time-shift property

```
m = 3
n = np.arange(N)
k = np.arange(N)

y = np.roll(x, m)
Y_direct = engine.transform(y)

phase = np.exp(-2j*np.pi*k*m/N)
Y_property = phase * X

print(np.allclose(Y_direct, Y_property, atol=1e-9))
```

Exam Tip

Shift questions are sign-error magnets. Do not guess the sign. First write the actual signal relation. If you used `np.roll(x, +m)`, that is a right shift and matches $x[((n - m))_N]$.

9.4 Frequency shift / modulation

The supplied property slide writes:

$$W_N^{-k_0 n} x[n] = e^{+j2\pi k_0 n/N} x[n] \longleftrightarrow X[((k - k_0))_N].$$

So multiplication by a complex sinusoid in time circularly shifts the DFT bins.

Listing 8: Verify modulation / frequency shift

```
k0 = 2
n = np.arange(N)

tone = np.exp(+2j*np.pi*k0*n/N)
y = tone * x

Y_direct = engine.transform(y)
Y_property = np.roll(X, k0) # Y[k] = X[(k-k0) mod N]

print(np.allclose(Y_direct, Y_property, atol=1e-9))
```

A common special case is $k_0 = N/2$ for even N :

$$e^{j\pi n} = (-1)^n.$$

Therefore multiplying every other sample by -1 shifts the spectrum by half the DFT clock.

Listing 9: Special case: alternating signs

```
y = x * ((-1) ** np.arange(N))
Y = engine.transform(y)
print(np.allclose(Y, np.roll(X, N//2), atol=1e-9))
```

9.5 Circular time reversal

$$x[((-n))_N] \longleftrightarrow X[((-k))_N].$$

The circular reversal is *not* simply `x[::-1]`. For $N = 8$, the correct index order is

$$0, 7, 6, 5, 4, 3, 2, 1.$$

Listing 10: Verify circular time reversal

```
idx = (-np.arange(N)) % N

x_rev = x[idx]
X_rev_direct = engine.transform(x_rev)

X_rev_property = X[idx]

print(np.allclose(X_rev_direct, X_rev_property, atol=1e-9))
```

9.6 Conjugation

$$x^*[n] \longleftrightarrow X^*[((-k))_N].$$

Listing 11: Verify conjugation property

```
z = np.array([1+2j, 3-1j, 2+0j, -1+4j, 0, 2j, 1, -3], dtype=complex)
Z = engine.transform(z)
idx = (-np.arange(N)) % N

left = engine.transform(np.conjugate(z))
right = np.conjugate(Z[idx])

print(np.allclose(left, right, atol=1e-9))
```

9.7 Conjugate symmetry for real signals

If $x[n]$ is real:

$$X[N - k] = X^*[k].$$

Equivalent statements:

$$|X[N - k]| = |X[k]|, \quad \angle X[N - k] = -\angle X[k].$$

Listing 12: Verify real-signal conjugate symmetry

```
x = np.array([3,5,2,0,1,2,4,1], dtype=float)
X = engine.transform(x)

idx = (-np.arange(N)) % N
print("full symmetry:",
      np.allclose(X[idx], np.conjugate(X), atol=1e-9))

for k in range(N):
    print(k, X[k], "<->", idx[k], X[idx[k]])
```

Useful corollary. If a real signal is also circularly even, then its DFT is real.

9.8 Parseval's relation

The supplied slide uses:

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

Listing 13: Verify Parseval

```
time_energy = np.sum(np.abs(x)**2)
freq_energy = np.sum(np.abs(X)**2) / N

print(time_energy)
print(freq_energy)
print(np.allclose(time_energy, freq_energy, atol=1e-9))
```

9.9 Duality

With this DFT convention:

$$x[n] \longleftrightarrow X[k]$$

implies

$$X[n] \longleftrightarrow N x[((-k))_N].$$

This is a good exam property because it checks whether you understand the scaling convention.

Listing 14: Verify DFT duality

```
x = np.array([1,2,3,4,0,0,0,0], dtype=complex)
X = engine.transform(x)
idx = (-np.arange(N)) % N

left = engine.transform(X)      # treat DFT bins as a new time signal
right = N * x[idx]

print(np.allclose(left, right, atol=1e-9))
```

9.10 Circular convolution theorem

For same-length N sequences:

$$(x \circledast h)[n] = \sum_{m=0}^{N-1} x[m] h[((n-m))_N]$$

and

$$x \circledast h \longleftrightarrow X[k] H[k].$$

The fastest implementation is:

$$x \circledast h = \text{IFFT}\{\text{FFT}(x)\text{FFT}(h)\}.$$

Listing 15: Circular convolution: direct and FFT methods

```
def circular_conv_direct(x, h):
    x = np.asarray(x, dtype=complex)
    h = np.asarray(h, dtype=complex)
    N = len(x)
    assert len(h) == N

    y = np.zeros(N, dtype=complex)
    for n in range(N):
        for m in range(N):
            y[n] += x[m] * h[(n-m) % N]
    return y
```

```
x = np.array([2,4,5,6,5,3,2,1], dtype=float)
h = np.array([1,2,1,0,0,0,0,0], dtype=float)

direct = circular_conv_direct(x, h)
spectral = engine.inverse(engine.transform(x) * engine.transform(h))

print(np.allclose(direct, spectral, atol=1e-9))
print(np.real_if_close(spectral))
```

9.11 Linear convolution via DFT/FFT

If x has length L and h has length M , linear convolution has length:

$$L + M - 1.$$

The crucial supplied slide rule is:

$$N \geq L + M - 1$$

so that circular convolution of the zero-padded sequences equals the linear convolution.

For radix-2:

$$N = \text{next_power_of_two}(L + M - 1).$$

Listing 16: Linear convolution with zero-padding

```
from transforms import FFTTransformer, next_power_of_two

def linear_conv_fft(x, h):
    x = np.asarray(x, dtype=complex)
    h = np.asarray(h, dtype=complex)

    needed = len(x) + len(h) - 1
    N = next_power_of_two(needed)

    xp = np.zeros(N, dtype=complex)
    hp = np.zeros(N, dtype=complex)
    xp[:len(x)] = x
    hp[:len(h)] = h

    fft = FFTTransformer()
    y = fft.inverse(fft.transform(xp) * fft.transform(hp))
    return y[:needed]

x = np.array([2,4,5,6,5,3,2,1])
h = np.array([1,2,1])

a = np.convolve(x, h)
b = linear_conv_fft(x, h)

print(np.allclose(a, b, atol=1e-9))
```

Important

Two different reasons for zero-padding appear in the course:

1. In the DFT/DTFT lecture, padding gives *finer samples of the same DTFT*.
2. In the convolution lecture, padding to at least $L + M - 1$ gives *room for the convolution tail so circular wrap does not corrupt the answer*.

The same code operation has two different interpretations depending on the problem.

9.12 Multiplication property

The property table also gives:

$$x[n]h[n] \longleftrightarrow \frac{1}{N}(X \circledast H)[k].$$

So multiplication in time corresponds to circular convolution in frequency.

Listing 17: Verify multiplication property

```
def circular_conv_same_length(a, b):
    N = len(a)
    out = np.zeros(N, dtype=complex)
    for k in range(N):
        for r in range(N):
            out[k] += a[r] * b[(k-r) % N]
    return out

x = np.array([1,2,3,4,0,0,0,0], dtype=complex)
h = np.array([2,1,0,3,1,0,0,0], dtype=complex)
X = engine.transform(x)
H = engine.transform(h)

left = engine.transform(x*h)
right = circular_conv_same_length(X, H) / N

print(np.allclose(left, right, atol=1e-9))
```

9.13 Circular correlation and shift detection

A practical correlation identity is:

$$r_{xy}[m] = \text{IDFT}\{X^*[k]Y[k]\}.$$

For a pure circular shift, the peak index gives the shift under the convention used below.

Listing 18: Circular shift detection by FFT correlation

```
def circular_shift_by_correlation(reference, shifted, engine):
    X = engine.transform(reference)
    Y = engine.transform(shifted)
    corr = engine.inverse(np.conjugate(X) * Y)
    return int(np.argmax(corr.real))

x = np.array([1,7,2,9,3,4,8,0], dtype=float)
true_shift = 3
y = np.roll(x, true_shift)

found = circular_shift_by_correlation(x, y, engine)
print("found =", found)
print("corrected =", np.roll(y, -found))
```

10 2-D transforms and image questions

The current assignment's `image_conv.py` uses separability: perform a 1-D transform on every row, then on every column.

10.1 2-D transform template

Listing 19: Assignment-style separable 2-D transform

```
def transform_2d(plane, engine):
    x = np.asarray(plane, dtype=np.complex128)

    row_done = np.empty_like(x)
    for r in range(x.shape[0]):
        row_done[r, :] = engine.transform(x[r, :])

    out = np.empty_like(row_done)
    for c in range(x.shape[1]):
        out[:, c] = engine.transform(row_done[:, c])

    return out

def inverse_2d(spectrum, engine):
    x = np.asarray(spectrum, dtype=np.complex128)

    col_done = np.empty_like(x)
    for c in range(x.shape[1]):
        col_done[:, c] = engine.inverse(x[:, c])

    out = np.empty_like(col_done)
    for r in range(x.shape[0]):
        out[r, :] = engine.inverse(col_done[r, :])

    return out
```

10.2 2-D linear convolution shape

If an image plane is $h \times w$ and the kernel is $k_h \times k_w$, the full linear convolution is

$$(h + k_h - 1) \times (w + k_w - 1).$$

For the assignment's radix-2 engine, each transform dimension is rounded up to the next power of two.

The spectral core is:

$$\text{IFFT}_2(\text{FFT}_2(\text{padded image}) \cdot \text{FFT}_2(\text{padded kernel})).$$

10.3 Color images

An RGB image is three independent 2-D signals. Convolve each channel with the same spatial kernel and stack the channels again.

Listing 20: Channel-wise image filtering

```
if image.ndim == 2:
    blurred = convolve_plane(image, kernel, engine)
else:
    blurred = np.stack([
        convolve_plane(image[:, :, c], kernel, engine)
        for c in range(image.shape[2])
    ], axis=2)
```

10.4 Circular image convolution and kernel centering

In circular convolution, the array origin is index $(0, 0)$. A normal blur kernel is conceptually centered in the middle of its array, so the assignment moves that center to the origin with `np.roll` before doing circular convolution.

Listing 21: Move kernel center to the circular-convolution origin

```
kp = np.zeros((H, W))
kp[:kh, :kw] = kernel
kp = np.roll(kp, -(kh // 2), axis=0)
kp = np.roll(kp, -(kw // 2), axis=1)
```

11 How to predict the new online from the old online and current assignment

Prediction / inference

The following ranking is an inference from the three supplied previous questions, the current assignment structure, and the properties lecture. It is not an official syllabus promise.

The old online already sampled three application families:

1. convolution plus carries (big integers);
2. weighted convolution (polynomials);
3. cross-correlation plus shifts (images).

The current assignment directly strengthens two of those families: big-integer spectral convolution and 2-D image convolution. The newly supplied properties lecture makes property verification especially likely.

11.1 Likelihood tiers

Tier	Question family	Why it is plausible
Very high	Circular vs linear convolution	Central application of the properties slide and both current assignment tasks.
Very high	Zero-padding correctness	Explicitly emphasized in both DFT/DTFT and convolution contexts.
Very high	Verify DFT property in two ways	The properties lecture is structured exactly around these equalities.
Very high	Shift / correlation / alignment	Already appeared previously and directly uses circular shift/conjugation.
High	Big integer / polynomial variant	Current Task A plus two old convolution-style questions.
High	2-D image filter / blur	Current Task B already provides reusable structure.
High	Parseval / conjugate symmetry	Easy 30-minute verification questions with clear numerical checks.
High	Modulation / frequency shift	A direct property-table question requiring only a few lines.
Medium	Time reversal / conjugation / duality	In the supplied property table but less application-heavy.
Medium	DFT vs FFT correctness and runtime	Current assignment explicitly compares both.
Medium	Implement/fix an FFT helper	The current assignment exposes bit reversal, butterflies, inverse scaling, and power-of-two checks.
Medium	Arbitrary-length transform	Bonus code exists; useful for genuine circular problems with non-power-of-two length.
Lower	Dense DTFT plotting / bin mapping	Present in theory, but less central to the old online application style.

11.2 A fast decision tree during the exam

1. Are you asked to combine two sequences by all pairwise index sums?
 - Yes: convolution.
 - If output must not wrap: pad to $L + M - 1$.

2. Are you asked to discover an unknown circular shift?
 - Use cross-correlation: $\text{IFFT}(\text{conj}(\text{FFT}(\text{reference})) * \text{FFT}(\text{observed}))$.
3. Are you asked to verify an identity?
 - Translate both sides into arrays and use `np.allclose`.
4. Is the data two-dimensional?
 - Reuse the assignment's row-then-column transform.
5. Is the FFT length not a power of two?
 - For linear convolution: pad farther.
 - For a true circular problem whose period must stay exactly N : use `ArbitraryLengthFFT`.

12 Twenty predicted practice questions with solutions

These are **practice questions**, not claims about the future paper. They are deliberately written in the style suggested by the previous online sheets: short scenario, one key property, code that can be adapted quickly.

12.1 Practice 1 – DFT/IDFT round trip and the missing $1/N$ bug

Possible question. Given a length-8 sequence, compute its DFT using your class, reconstruct it using the inverse, and verify the reconstruction. Then explain what happens if the inverse forgets to divide by N .

Recognition. This tests the transform convention itself.

Logic. With the supplied convention,

$$\text{IDFT}(\text{DFT}(x)) = x.$$

If the inverse omits $1/N$, the reconstructed sequence is exactly Nx apart from floating-point error.

Listing 22: Practice 1 solution

```
import numpy as np
from transforms import DFTAnalyzer, FFTTransformer

x = np.array([1,2,3,4,2,1,0,-1], dtype=float)

for engine in [DFTAnalyzer(), FFTTransformer()]:
    X = engine.transform(x)
    xr = engine.inverse(X)

    print(engine.name)
    print("max reconstruction error:",
          np.max(np.abs(xr - x)))
    print("match:",
          np.allclose(xr, x, atol=1e-9))

# If someone wrote inverse without /N:
N = len(x)
wrong = N * x
print("wrong inverse would scale by", N)
```

Pitfalls. Do not round a general Fourier reconstruction unless the expected signal is known to be integer-valued. For numerical verification, use `np.allclose`.

12.2 Practice 2 – Verify linearity in two independent ways

Possible question. For given x, y, a, b , verify

$$\mathcal{F}\{ax + by\} = aX + bY.$$

Logic. Compute the DFT of the mixed signal for the left side. Separately transform x and y , then combine spectra for the right side.

Listing 23: Practice 2 solution

```
import numpy as np
from transforms import FFTTransformer

engine = FFTTransformer()
x = np.array([1,2,0,1,3,0,2,1], dtype=float)
y = np.array([0,1,4,2,0,3,1,2], dtype=float)
a, b = 3.0, -0.5

left = engine.transform(a*x + b*y)
right = a*engine.transform(x) + b*engine.transform(y)

print("match =", np.allclose(left, right, atol=1e-9))
print("max error =", np.max(np.abs(left-right)))
```

Likely variation. They may ask you to write a function `verify_linearity(x,y,a,b,engine)` that returns a Boolean and maximum error.

12.3 Practice 3 – Circular time shift: compute spectrum without transforming the shifted signal

Possible question. You already know $X[k]$. The signal is circularly shifted right by m . Compute the new spectrum using only $X[k]$, then verify against a direct FFT.

Key property.

$$x[((n - m))_N] \longleftrightarrow e^{-j2\pi km/N} X[k].$$

Listing 24: Practice 3 solution

```
import numpy as np
from transforms import FFTTransformer

x = np.array([3,1,4,1,5,9,2,6], dtype=float)
m = 3
N = len(x)
k = np.arange(N)
engine = FFTTransformer()

X = engine.transform(x)

shifted = np.roll(x, m)
Y_direct = engine.transform(shifted)

Y_property = np.exp(-2j*np.pi*k*m/N) * X

print(np.allclose(Y_direct, Y_property, atol=1e-9))
```

What to explain. The magnitude spectrum does not change because the multiplying phase factor has magnitude 1. Only the phase changes.

12.4 Practice 4 – Modulation shifts the spectrum

Possible question. Multiply $x[n]$ by $e^{j2\pi k_0 n/N}$ and prove that the DFT shifts by k_0 bins.

$$e^{j2\pi k_0 n/N} x[n] \longleftrightarrow X[(k - k_0))_N].$$

Listing 25: Practice 4 solution

```
import numpy as np
from transforms import FFTTransformer

x = np.array([2,4,5,6,5,3,2,1], dtype=float)
N = len(x)
k0 = 2
n = np.arange(N)
engine = FFTTransformer()

X = engine.transform(x)
modulated = x * np.exp(2j*np.pi*k0*n/N)

left = engine.transform(modulated)
right = np.roll(X, k0)

print(np.allclose(left, right, atol=1e-9))
```

Useful special case. For $k_0 = N/2$, the multiplier is $(-1)^n$. This can convert a low-pass-shaped spectrum into a high-pass-shaped one by moving the low-frequency cluster by half the DFT period.

12.5 Practice 5 – Time reversal and conjugation

Possible question. For a complex sequence, verify both:

$$x[((-n))_N] \longleftrightarrow X[((-k))_N],$$

and

$$x^*[n] \longleftrightarrow X^*[((-k))_N].$$

Listing 26: Practice 5 solution

```
import numpy as np
from transforms import FFTTransformer

x = np.array([1+1j, 2-3j, 4, 1j, -2, 3+2j, 0, 5], dtype=complex)
N = len(x)
idx = (-np.arange(N)) % N
engine = FFTTransformer()
X = engine.transform(x)

# Reversal
A = engine.transform(x[idx])
B = X[idx]
print("reversal:", np.allclose(A, B, atol=1e-9))

# Conjugation
C = engine.transform(np.conjugate(x))
D = np.conjugate(X[idx])
print("conjugation:", np.allclose(C, D, atol=1e-9))
```

Pitfall. `x[::-1]` gives indices $N-1, \dots, 0$. Circular reversal keeps index 0 fixed, so use `x[(-np.arange(N)) % N]`.

12.6 Practice 6 – Reconstruct the missing half of the DFT of a real signal

Possible question. You are given the lower DFT bins of a real length- N signal. Fill the upper bins using conjugate symmetry.

For a real signal:

$$X[N - k] = X^*[k].$$

For even N , the DC bin $k = 0$ and Nyquist bin $k = N/2$ are self-paired and must be real up to numerical error.

Listing 27: Practice 6 solution

```
import numpy as np

def complete_real_spectrum(first_half, N):
    """first_half must contain bins 0..N//2 for even N."""
    X = np.zeros(N, dtype=complex)
    X[:N//2 + 1] = first_half

    for k in range(1, N//2):
        X[N-k] = np.conjugate(X[k])

    return X

N = 8
known = np.array([
    10+0j, 2+3j, -1+4j, 5-2j, 7+0j
])
X = complete_real_spectrum(known, N)

print(X)
print(np.allclose(X[(-np.arange(N)) % N],
                  np.conjugate(X)))
```

Likely variation. They may ask only to verify magnitude symmetry:

$$|X[k]| = |X[N - k]|.$$

12.7 Practice 7 – Parseval: compare energy in time and frequency

Possible question. Calculate signal energy directly and from the DFT. Show equality.

Listing 28: Practice 7 solution

```
import numpy as np
from transforms import FFTTransformer

x = np.array([3,5,2,0,1,2,4,1], dtype=float)
N = len(x)
engine = FFTTransformer()
X = engine.transform(x)

Et = np.sum(np.abs(x)**2)
Ef = np.sum(np.abs(X)**2) / N

print("time energy =", Et)
print("freq energy =", Ef)
print("match =", np.allclose(Et, Ef, atol=1e-9))
```

What to explain. The DFT is a change of basis built from orthogonal complex exponentials. Energy is re-bookkept among the frequency bins, not created or destroyed.

12.8 Practice 8 – Verify duality

Possible question. Given $x[n] \leftrightarrow X[k]$, treat $X[k]$ itself as a length- N input signal and DFT it. Verify:

$$\mathcal{F}\{X[n]\} = Nx[((-k))_N].$$

Listing 29: Practice 8 solution

```
import numpy as np
from transforms import FFTTransformer

x = np.array([1,2,3,4,0,1,0,-1], dtype=complex)
N = len(x)
idx = (-np.arange(N)) % N
engine = FFTTransformer()

X = engine.transform(x)
left = engine.transform(X)
right = N * x[idx]

print(np.allclose(left, right, atol=1e-9))
```

Pitfall. The factor N depends on the course's DFT/IDFT scaling convention. Because the forward DFT has no $1/N$, duality contains N .

12.9 Practice 9 – Circular convolution: direct loops versus FFT

Possible question. Compute $x \circledast h$ in two ways: direct modular indexing and FFT/IFFT. Show that they match.

Listing 30: Practice 9 solution

```
import numpy as np
from transforms import FFTTransformer

def circular_conv_direct(x, h):
    N = len(x)
    y = np.zeros(N, dtype=complex)
    for n in range(N):
        for m in range(N):
            y[n] += x[m] * h[(n-m) % N]
    return y

x = np.array([2,4,5,6,5,3,2,1], dtype=float)
h = np.array([1,2,1,0,0,0,0,0], dtype=float)
engine = FFTTransformer()

y1 = circular_conv_direct(x, h)
y2 = engine.inverse(engine.transform(x) * engine.transform(h))

print(np.real_if_close(y1))
print(np.real_if_close(y2))
print(np.allclose(y1, y2, atol=1e-9))
```

What they may ask next. Compare this circular result with `np.convolve` and explain why the first few values differ: wrapped tail samples have folded into the front.

12.10 Practice 10 – Demonstrate that zero-padding turns circular convolution into linear convolution

Possible question. Compute:

1. an unpadded FFT convolution of length $N = L$;
2. a padded FFT convolution with $N \geq L + M - 1$;
3. `np.convolve`.

Show that only the padded result matches the linear convolution.

Listing 31: Practice 10 solution

```
import numpy as np
from transforms import FFTTransformer, next_power_of_two

x = np.array([2,4,5,6,5,3,2,1], dtype=float)
h = np.array([1,2,1], dtype=float)
fft = FFTTransformer()

# Unpadded circular convolution: pad h only to len(x)
hp = np.zeros(len(x))
hp[:len(h)] = h
circular = fft.inverse(fft.transform(x) * fft.transform(hp)).real

# Correct linear convolution
needed = len(x) + len(h) - 1
N = next_power_of_two(needed)
xp = np.zeros(N)
hp2 = np.zeros(N)
xp[:len(x)] = x
hp2[:len(h)] = h

linear_fft = fft.inverse(fft.transform(xp)*fft.transform(hp2)).real[:needed]
linear_direct = np.convolve(x, h)

print("circular =", np rint(circular).astype(int))
print("linear   =", np rint(linear_fft).astype(int))
print("matches np.convolve =",
      np.allclose(linear_fft, linear_direct, atol=1e-9))
```

Core explanation. Padding zeros are not extra signal information here; they are empty positions that prevent the convolution tail from wrapping onto the beginning.

12.11 Practice 11 – Verify the multiplication property

Possible question. For same-length x, h , verify:

$$\mathcal{F}\{x[n]h[n]\} = \frac{1}{N}(X \circledast H)[k].$$

Listing 32: Practice 11 solution

```
import numpy as np
from transforms import FFTTransformer

def circ_conv(a, b):
    N = len(a)
    out = np.zeros(N, dtype=complex)
    for k in range(N):
        for r in range(N):
            out[k] += a[r] * b[(k-r) % N]
    return out

x = np.array([1,2,3,4,0,0,0,0], dtype=complex)
h = np.array([2,1,0,3,1,0,0,0], dtype=complex)
N = len(x)
engine = FFTTransformer()

X = engine.transform(x)
H = engine.transform(h)

left = engine.transform(x*h)
right = circ_conv(X, H) / N

print(np.allclose(left, right, atol=1e-9))
```

Why this is a good online question. It is the mirror image of the convolution theorem and tests whether you remember the $1/N$.

12.12 Practice 12 – Big-integer multiplication, but with a different base

Possible question. Instead of one decimal digit per coefficient, group two digits per limb (base 100) or four digits per limb (base 10^4), as in the current assignment. Multiply with FFT and reconstruct the decimal answer.

Recognition. Only the base and carry rule change. The convolution is identical.

Listing 33: Practice 12 solution

```
import numpy as np
from transforms import FFTTransformer, next_power_of_two

BASE_DIGITS = 2
BASE = 10 ** BASE_DIGITS

def to_limbs(text):
    text = text.lstrip("0") or "0"
    limbs = []
    for end in range(len(text), 0, -BASE_DIGITS):
        start = max(0, end-BASE_DIGITS)
        limbs.append(int(text[start:end]))
    return np.array(limbs, dtype=np.int64)

def from_limbs(vals):
    vals = [int(v) for v in vals]
    i = 0
    while i < len(vals):
        q, r = divmod(vals[i], BASE)
        vals[i] = r
        if q:
            if i+1 == len(vals):
                vals.append(q)
            else:
                vals[i+1] += q
        i += 1

    while len(vals) > 1 and vals[-1] == 0:
        vals.pop()

    return str(vals[-1]) + ".".join(
        str(v).zfill(BASE_DIGITS)
        for v in reversed(vals[:-1])
    )

def multiply(a_text, b_text):
    a = to_limbs(a_text)
    b = to_limbs(b_text)

    needed = len(a)+len(b)-1
    N = next_power_of_two(needed)

    ap = np.zeros(N, complex)
    bp = np.zeros(N, complex)
    ap[:len(a)] = a
    bp[:len(b)] = b

    fft = FFTTransformer()
    c = fft.inverse(fft.transform(ap)*fft.transform(bp))
    c = np rint(c.real[:needed]).astype(np.int64)

    return from_limbs(c)

print(multiply("123456789", "987654321"))
```

Connection to current assignment. This is almost exactly `to_limbs`, `multiply_transform`, and `from_limbs` from `bigmul.py`.

12.13 Practice 13 – Weighted polynomial convolution with a changed weighting rule

Possible question. Given polynomial coefficients and a weight sequence, multiply each coefficient of P by its weight before multiplying by Q . Or, a variation may weight Q instead.

Strategy. Do the non-convolution operation first (elementwise weighting), then use the standard convolution template.

Listing 34: Practice 13 solution

```
import numpy as np
from transforms import FFTTransformer, next_power_of_two

def weighted_product(P, Q, W):
    P = np.asarray(P, dtype=np.int64)
    Q = np.asarray(Q, dtype=np.int64)
    W = np.asarray(W, dtype=np.int64)

    A = P * W
    needed = len(A) + len(Q) - 1
    N = next_power_of_two(needed)

    ap = np.zeros(N, complex)
    bp = np.zeros(N, complex)
    ap[:len(A)] = A
    bp[:len(Q)] = Q

    fft = FFTTransformer()
    r = fft.inverse(fft.transform(ap)*fft.transform(bp))
    return np rint(r.real[:needed]).astype(np.int64)

print(weighted_product([1,3,2], [4,1], [3,2,1]))
```

Exam adaptation. If the formula contains something like $i p_i$ or $(-1)^i p_i$, construct that modified coefficient array first. Then ask whether the remaining sum is a convolution.

12.14 Practice 14 – Detect a circular shift of one 1-D signal

Possible question. A known template x was circularly shifted by an unknown amount to produce y . Recover the shift with FFT-based cross-correlation and restore x .

Listing 35: Practice 14 solution

```
import numpy as np
from transforms import FFTTransformer

def find_shift(reference, observed):
    fft = FFTTransformer()
    X = fft.transform(reference)
    Y = fft.transform(observed)
    corr = fft.inverse(np.conjugate(X) * Y).real
    return int(np.argmax(corr))

x = np.array([3,1,4,1,5,9,2,6], dtype=float)
y = np.roll(x, 5)

s = find_shift(x, y)
restored = np.roll(y, -s)

print("shift =", s)
print("match =", np.allclose(restored, x))
```

Robustness variation. If the observed signal has a different DC level, subtract the mean from both signals before the FFT.

12.15 Practice 15 – Correct row-wise image distortion

Possible question. Exactly like the previous Section C, but perhaps the image is grayscale, or only every second row is shifted, or the shift range is bounded.

Reuse. Treat each row independently and reuse Practice 14. For RGB, sum correlations across channels.

Listing 36: Practice 15 core loop

```
def correct_rows(original, shifted, engine):
    corrected = shifted.copy()
    shifts = []

    for r in range(original.shape[0]):
        corr = 0

        # grayscale: original.ndim == 2
        if original.ndim == 2:
            a = original[r] - np.mean(original[r])
            b = shifted[r] - np.mean(shifted[r])
            A = engine.transform(a)
            B = engine.transform(b)
            corr = engine.inverse(np.conjugate(A)*B).real

        # RGB
        else:
            corr = np.zeros(original.shape[1])
            for c in range(original.shape[2]):
                a = original[r,:,c] - np.mean(original[r,:,c])
                b = shifted[r,:,c] - np.mean(shifted[r,:,c])
                A = engine.transform(a)
                B = engine.transform(b)
                corr += engine.inverse(np.conjugate(A)*B).real

            s = int(np.argmax(corr))
            shifts.append(s)
            corrected[r] = np.roll(shifted[r], -s, axis=0)

    return corrected, shifts
```

Engine choice. Use ArbitraryLengthFFT if the image width is not a power of two and the correlation must remain circular over exactly that width.

12.16 Practice 16 – Blur a small image with a kernel using your assignment

Possible question. Given a grayscale image array and a 3×3 kernel, compute linear convolution in the frequency domain, return an output with the original image dimensions, and verify against direct convolution.

Recognition. This is current Task B with a smaller fixed kernel.

Listing 37: Practice 16 solution

```
import numpy as np
from transforms import FFTTransformer, next_power_of_two

def transform_2d(x, engine):
    x = np.asarray(x, dtype=complex)
    temp = np.empty_like(x)
    for r in range(x.shape[0]):
        temp[r] = engine.transform(x[r])

    out = np.empty_like(temp)
    for c in range(x.shape[1]):
        out[:,c] = engine.transform(temp[:,c])
    return out

def inverse_2d(X, engine):
    X = np.asarray(X, dtype=complex)
    temp = np.empty_like(X)
    for c in range(X.shape[1]):
        temp[:,c] = engine.inverse(X[:,c])

    out = np.empty_like(temp)
    for r in range(X.shape[0]):
        out[r] = engine.inverse(temp[r])
    return out

def convolve2d_same(image, kernel):
    h,w = image.shape
    kh,kw = kernel.shape

    H = next_power_of_two(h+kh-1)
    W = next_power_of_two(w+kw-1)

    ip = np.zeros((H,W))
    kp = np.zeros((H,W))
    ip[:h,:w] = image
    kp[:kh,:kw] = kernel

    fft = FFTTransformer()
    full = inverse_2d(
        transform_2d(ip, fft) * transform_2d(kp, fft),
        fft
    ).real

    return full[kh//2:kh//2+h, kw//2:kw//2+w]
```

Likely variation. They may ask you to compare linear blur against circular wraparound blur and identify edge artifacts.

12.17 Practice 17 – Build a simple frequency-domain filter

Possible question. Given a length- N signal and a cutoff bin K , keep only low-frequency DFT bins and reconstruct the filtered signal.

Key bin fact. For a real signal, positive frequencies occupy low bins and the corresponding negative frequencies occupy the high bins. A symmetric low-pass mask must preserve both sides.

Listing 38: Practice 17 solution

```
import numpy as np
from transforms import FFTTransformer

x = np.array([0,1,0,1,0,1,4,-2,3,1,0,2,1,0,-1,2], dtype=float)
N = len(x)
K = 2
fft = FFTTransformer()

X = fft.transform(x)

mask = np.zeros(N)
mask[:K+1] = 1           # DC through +K
mask[-K:] = 1           # -K through -1

Y = X * mask
y = fft.inverse(Y).real

print("filtered signal =", y)
```

Why symmetry matters. If you keep a positive-frequency bin but delete its conjugate partner, the inverse can become complex even though the original signal was real.

12.18 Practice 18 – Zero-padding as finer samples of the same DTFT

Possible question. Compute the DFT of a short sequence at length N and again after padding to $4N$. Explain why the second spectrum has more points but not more underlying information.

Theory. Appending zeros does not change

$$X(e^{j\omega}) = \sum_n x[n]e^{-j\omega n}.$$

It only samples that same DTFT at a smaller frequency spacing.

Listing 39: Practice 18 solution

```
import numpy as np

x = np.array([1,2,3,4], dtype=float)

# Using NumPy here only to make the sampling idea compact.
# The same experiment can be done with DFTAnalyzer.
X4 = np.fft.fft(x, 4)
X16 = np.fft.fft(x, 16)

print("4-point bins :", X4)
print("16-point bins:", X16)

# The old 4 bins are every 4th bin of the 16-point DFT.
print(np.allclose(X4, X16[::4]))
```

Do not say that zero-padding increases the true frequency resolution of the captured data. The supplied lecture explicitly describes it as finer samples / interpolation of the same DTFT.

12.19 Practice 19 – Compare naive DFT and FFT results and complexity

Possible question. For power-of-two lengths, verify that your `DFTAnalyzer` and `FFTTransformer` return the same values. Time both for increasing N .

Listing 40: Practice 19 solution

```
import time
import numpy as np
from transforms import DFTAnalyzer, FFTTransformer

rng = np.random.default_rng(0)

for N in [32, 64, 128, 256, 512]:
    x = rng.normal(size=N)

    dft = DFTAnalyzer()
    fft = FFTTransformer()

    t0 = time.perf_counter()
    Xd = dft.transform(x)
    td = time.perf_counter() - t0

    t0 = time.perf_counter()
    Xf = fft.transform(x)
    tf = time.perf_counter() - t0

    print(N,
          "match", np.allclose(Xd, Xf, atol=1e-9),
          "DFT", td,
          "FFT", tf)
```

What to explain. Both compute the same transform. The direct method grows as $O(N^2)$; radix-2 FFT grows as $O(N \log N)$.

Non-power-of-two variation. `FFTTransformer` should raise an error. Use the current assignment's `ArbitraryLengthFFT`, or pad if changing the length is mathematically allowed.

12.20 Practice 20 – FFT implementation / debugging question

Possible question A. A provided inverse FFT produces answers exactly N times too large. Fix it.

Answer. Add:

```
if inverse:
    a /= N
```

Possible question B. The FFT gives scrambled output even though each butterfly is correct. A likely missing step is the DIT bit-reversal permutation.

Possible question C. Implement a helper returning the bit-reversed order for $N = 2^v$.

Listing 41: Practice 20: explicit bit reversal

```
def bit_reversed_indices(N):
    if N <= 0 or (N & (N-1)):
        raise ValueError("N must be a power of two")

    bits = N.bit_length() - 1
    out = []

    for i in range(N):
        r = 0
        x = i
        for _ in range(bits):
            r = (r << 1) | (x & 1)
            x >>= 1
        out.append(r)

    return out

print(bit_reversed_indices(8))
# [0, 4, 2, 6, 1, 5, 3, 7]
```

Possible question D. Explain one butterfly:

$$u = a[\text{top}], \quad v = W_m^k a[\text{bottom}],$$

then write:

$$a[\text{top}] = u + v, \quad a[\text{bottom}] = u - v.$$

Exam Tip

If they ask you to modify the assignment FFT rather than solve a new application, first look for one of four classic bugs: (1) wrong sign in the exponent, (2) missing $1/N$ in inverse, (3) missing/incorrect bit reversal, (4) non-power-of-two length passed to radix-2.

13 DTFT essentials that may still appear in an online

The supplied first lecture contains the DTFT definition and inverse, but the separate supplied properties lecture is specifically a *DFT properties* lecture. So the lab-ready property cookbook above is source-grounded in the DFT slide. This section covers the DTFT material that is explicitly present in Lecture 1 and is useful for coding questions.

13.1 Evaluate the DTFT numerically on a dense grid

For a finite sequence:

$$X(e^{j\omega}) = \sum_{n=0}^{N-1} x[n]e^{-j\omega n}.$$

Listing 42: Dense-grid DTFT evaluation

```
import numpy as np

def dtft(x, omega):
    x = np.asarray(x, dtype=complex).reshape(-1)
    omega = np.asarray(omega, dtype=float).reshape(-1)
    n = np.arange(len(x))
    return np.exp(-1j * omega[:, None] * n[None, :]) @ x

x = np.array([1,2,3,4], dtype=float)
omega = np.linspace(-np.pi, np.pi, 1001)
Xw = dtft(x, omega)

print(Xw.shape)    # one DTFT value per omega
```

13.2 Verify 2π periodicity of the DTFT

The supplied lecture proves:

$$X(e^{j(\omega+2\pi)}) = X(e^{j\omega}).$$

Listing 43: DTFT periodicity

```
omega = np.linspace(-np.pi, np.pi, 200)
A = dtft(x, omega)
B = dtft(x, omega + 2*np.pi)

print(np.allclose(A, B, atol=1e-9))
```

13.3 Verify that the DFT samples the DTFT

The DFT bins occur at:

$$\omega_k = \frac{2\pi k}{N}.$$

Listing 44: DFT as samples of DTFT

```
from transforms import DFTAnalyzer

x = np.array([1,2,3,4], dtype=float)
N = len(x)
omega_k = 2*np.pi*np.arange(N)/N

X_from_dtft = dtft(x, omega_k)
X_dft = DFTAnalyzer().transform(x)

print(np.allclose(X_from_dtft, X_dft, atol=1e-9))
```

13.4 Map DFT bins to physical frequency

If the sample rate is f_s :

$$\Delta f = \frac{f_s}{N}.$$

For the usual unshifted DFT storage:

$$f_k = \begin{cases} kf_s/N, & 0 \leq k \leq N/2, \\ (k-N)f_s/N, & N/2 < k \leq N-1. \end{cases}$$

Listing 45: DFT bin to Hz

```
def bin_frequency(k, N, fs):
    if k <= N//2:
        return k * fs / N
    return (k-N) * fs / N

N = 8
fs = 8000
print([bin_frequency(k, N, fs) for k in range(N)])
# [0,1000,2000,3000,4000,-3000,-2000,-1000]
```

13.5 Zero-padding and old-bin consistency

If an N -sample signal is padded to $M = qN$ samples, the original N DFT sample frequencies are still present at every q -th bin of the M -point DFT.

Listing 46: Old DFT bins inside a zero-padded DFT

```
x = np.array([1,2,3,4], dtype=float)
N = len(x)
M = 4*N

Xd = np.fft.fft(x, N)
Xp = np.fft.fft(x, M)

print(np.allclose(Xd, Xp[:M//N]))
```

14 How to reuse the assignment OOP structure in a new online file

The current assignment deliberately centralizes transforms behind a common interface:

```
engine.transform(x)
engine.inverse(X)
```

That means a new online function should usually accept an `engine` argument instead of hard-coding one transform implementation.

14.1 Pattern 1 – property function with an engine parameter

Listing 47: Engine-agnostic property function

```
def verify_parseval(x, engine):
    x = np.asarray(x, dtype=complex)
    X = engine.transform(x)
    left = np.sum(np.abs(x)**2)
    right = np.sum(np.abs(X)**2) / len(x)
    return np.allclose(left, right, atol=1e-9)
```

Now the same online code can run with:

```
verify_parseval(x, DFTAnalyzer())
verify_parseval(x, FFTTransformer())
verify_parseval(x, ArbitraryLengthFFT())
```

14.2 Pattern 2 – choose engine by a string

Listing 48: Assignment-style engine selection

```
from transforms import DFTAnalyzer, FFTTransformer, ArbitraryLengthFFT

def make_engine(name):
    return {
        "dft": DFTAnalyzer,
        "fft": FFTTransformer,
        "arbitrary": ArbitraryLengthFFT,
    }[name]()
```

14.3 Pattern 3 – add a new class without changing the application

If the online asks you to create a class such as a correlator, keep transform responsibility in the engine.

Listing 49: Small OOP wrapper around the assignment engine

```
class CircularCorrelator:
    def __init__(self, engine):
        self.engine = engine

    def correlate(self, x, y):
        X = self.engine.transform(x)
        Y = self.engine.transform(y)
        return self.engine.inverse(np.conjugate(X) * Y)

    def find_shift(self, reference, observed):
        r = self.correlate(reference, observed)
        return int(np.argmax(r.real))
```

14.4 Pattern 4 – modify an existing assignment function minimally

If the online says “change Task B so that it performs correlation instead of convolution”, do not rewrite the transform core. Only replace the spectral multiplication:

$$X[k]H[k]$$

with the expression needed by the new operation, for example:

$$X^*[k]Y[k]$$

for the correlation convention used in this guide.

Exam Tip

The strongest exam habit is to preserve working code. Change the smallest possible layer: data preparation, spectral multiplication, or post-processing. Do not rewrite the FFT unless the question explicitly asks for the FFT itself.

15 Lab-ready copy blocks

This section is intentionally repetitive. It exists so that during a timed online you can search the PDF for a phrase and copy a compact block.

15.1 Block A – standard imports

Listing 50: Imports

```
import numpy as np

from transforms import (
    DFTAnalyzer,
    FFTTransformer,
    ArbitraryLengthFFT,
    next_power_of_two,
)
```

15.2 Block B – safe linear convolution

Listing 51: Copy block: linear convolution

```
def fft_linear_convolution(a, b):
    a = np.asarray(a, dtype=complex).reshape(-1)
    b = np.asarray(b, dtype=complex).reshape(-1)

    needed = len(a) + len(b) - 1
    N = next_power_of_two(needed)

    ap = np.zeros(N, complex)
    bp = np.zeros(N, complex)
    ap[:len(a)] = a
    bp[:len(b)] = b

    fft = FFTTransformer()
    y = fft.inverse(fft.transform(ap) * fft.transform(bp))
    return y[:needed]
```

15.3 Block C – same-length circular convolution

Listing 52: Copy block: circular convolution

```
def fft_circular_convolution(a, b, engine):
    a = np.asarray(a, dtype=complex).reshape(-1)
    b = np.asarray(b, dtype=complex).reshape(-1)
    if len(a) != len(b):
        raise ValueError("circular convolution requires same length")
    return engine.inverse(engine.transform(a) * engine.transform(b))
```

15.4 Block D – circular correlation and shift

Listing 53: Copy block: correlation

```
def circular_correlation(a, b, engine):
    A = engine.transform(a)
    B = engine.transform(b)
    return engine.inverse(np.conjugate(A) * B)

def detect_shift(reference, observed, engine):
    corr = circular_correlation(reference, observed, engine)
    return int(np.argmax(corr.real))
```

15.5 Block E – property verification

Listing 54: Copy block: numerical verification

```
def report_match(left, right, tol=1e-9):
    left = np.asarray(left)
    right = np.asarray(right)
    err = np.max(np.abs(left-right))
    print("max error =", err)
    print("match =", np.allclose(left, right, atol=tol))
    return err
```

15.6 Block F – circular reversal indices

Listing 55: Copy block: circular reversal

```
idx = (-np.arange(N)) % N
x_rev = x[idx]
X_rev = X[idx]
```

15.7 Block G – right circular shift and its phase

Listing 56: Copy block: shift property

```
m = 3
k = np.arange(N)

shifted = np.roll(x, m)
shifted_spectrum_from_property = (
    np.exp(-2j*np.pi*k*m/N) * X
)
```

15.8 Block H – modulation

Listing 57: Copy block: modulation property

```
k0 = 2
n = np.arange(N)

modulated = x * np.exp(+2j*np.pi*k0*n/N)
shifted_spectrum = np.roll(X, k0)
```

15.9 Block I – Parseval

Listing 58: Copy block: Parseval

```
X = engine.transform(x)
Et = np.sum(np.abs(x)**2)
Ef = np.sum(np.abs(X)**2) / len(x)
print(np.allclose(Et, Ef, atol=1e-9))
```

15.10 Block J – real-signal conjugate symmetry

Listing 59: Copy block: conjugate symmetry

```
N = len(x)
idx = (-np.arange(N)) % N
X = engine.transform(x)

print(np.allclose(
    X[idx],
    np.conjugate(X),
    atol=1e-9
))
```

15.11 Block K – integer cleanup after IFFT

Listing 60: Copy block: integer-valued convolution cleanup

```
values = np rint(result.real).astype(np.int64)
```

Use that only when the exact mathematical answer should be integer-valued. Do not indiscriminately round a filtered image or a general signal.

16 Python and NumPy syntax cheat sheet for this online

16.1 Array creation and type

```
x = np.array([1,2,3,4], dtype=float)
z = np.asarray(x, dtype=np.complex128)
z = z.reshape(-1)           # force 1-D
N = z.size
```

16.2 Zeros and padding

```
p = np.zeros(N, dtype=np.complex128)
p[:len(x)] = x
```

16.3 Complex numbers

```
1j                # sqrt(-1)
np.conjugate(X)    # complex conjugate
X.real
X.imag
np.abs(X)          # magnitude
np.angle(X)        # phase in radians
```

16.4 Complex exponentials

```
np.exp(-2j*np.pi*k*n/N)
np.exp(+2j*np.pi*k0*n/N)
```

16.5 Useful indexing

```
np.arange(N)      # 0,1,...,N-1
(-np.arange(N)) % N  # circular reversal indices
np.roll(x, +m)     # circular right shift
np.roll(x, -m)     # circular left shift
x[::-1]           # ordinary array reversal, NOT circular reversal
```

16.6 Comparing numerical results

```
np.allclose(a, b, atol=1e-9)
np.max(np.abs(a-b))
```

16.7 Finding a correlation peak

```
shift = int(np.argmax(corr.real))
```

16.8 Rounding values known to be integers

```
c = np rint(c.real).astype(np.int64)
```

16.9 2-D / image indexing

```
image.shape      # (H,W) or (H,W,C)
image[r, :]      # row r
image[:, c]      # column c
image[:, :, channel] # one RGB channel
np.stack(arrays, axis=2) # rebuild RGB image
```

16.10 Power-of-two test

```
is_power_of_two = (N > 0) and ((N & (N-1)) == 0)
```

16.11 Bit shifts used in the FFT code

```
p <<= 1          # p *= 2  
bit >>= 1        # bit /= 2
```

17 Common failure modes and how to diagnose them

Symptom	Most likely cause / fix
IFFT result is exactly N times too large	Missing division by N in inverse.
FFT raises power-of-two error	Use next power of two if padding is mathematically allowed; otherwise use <code>ArbitraryLengthFFT</code> .
Linear convolution first samples are corrupted	Insufficient zero-padding; need $N \geq L + M - 1$.
Circular shift property has correct magnitude but wrong phase	Sign of m or the exponential is reversed. Check whether you used <code>np.roll(x, +m)</code> .
Time-reversal property fails	Used <code>x[::-1]</code> instead of circular reversal $x[(-n) \bmod N]$.
Real-signal inverse unexpectedly complex	Broke conjugate symmetry in frequency-domain manipulation.
Integer convolution returns 14.999999998	Normal floating-point error; use <code>np rint</code> before integer conversion.
Integer convolution gives completely wrong low digits	Circular wrap due to insufficient padding, or carry propagation is wrong.
Image blur is shifted spatially	Kernel origin/centering is wrong.
Image circular blur has weird opposite-edge influence	That is the expected wraparound of circular convolution.
Cross-correlation detects opposite sign	Correlation convention changed; test once with a known <code>np.roll</code> and decide whether to undo by $+s$ or $-s$.
Cross-correlation fails on a flat row	A constant row contains no shift information; any circular shift is identical.
Exact image comparison fails after correcting a JPEG	JPEG may have altered pixel values through lossy compression. Compare geometry/error, not only <code>array_equal</code> .
DFT and FFT differ slightly around 10^{-12}	Normal numerical error; use <code>np.allclose</code> .
DFT and FFT differ significantly	Check exponent sign, bit reversal, twiddle indexing, and inverse scaling.

18 Complexity cheat sheet

Operation	Typical cost	Comment
Direct N -point DFT	$O(N^2)$	matrix/sum definition
Radix-2 FFT	$O(N \log N)$	N power of two
Bluestein arbitrary FFT	$O(N \log N)$	converts DFT to convolution
Direct 1-D convolution	$O(LM)$	two nested loops
FFT linear convolution	roughly $O(N \log N)$	$N \geq L + M - 1$
Direct 2-D $K \times K$ filtering of $N \times N$ image	roughly $O(N^2 K^2)$	four-loop spatial form
Separable row-column 2-D FFT	roughly $O(HW(\log H + \log W))$	plus spectral multiply

19 What to memorize versus what to search in this PDF

Memorize these

1. DFT: negative exponent; IDFT: positive exponent and divide by N .
2. FFT computes the same DFT faster.
3. Circular shift right by m : `np.roll(x,m)`.
4. Convolution theorem: FFT, multiply, IFFT.
5. Linear convolution needs $N \geq L + M - 1$.
6. Correlation shift detector: `IFFT(conj(FFT(reference)) * FFT(observed))`.
7. Real signal: conjugate-symmetric DFT.
8. Parseval: time energy = frequency energy divided by N .

Search this PDF for these

- “modulation”
- “circular reversal”
- “duality”
- “weighted polynomial”
- “carry propagation”
- “row-wise image”
- “2-D transform”
- “multiplication property”
- “bit reversal”
- “zero-padding”

20 Extra mini-prompts for fast drill

Try to solve these without looking at the answers first. Most can be done by adapting one copy block.

1. Shift a signal left by 2 and predict its DFT from $X[k]$.
2. Given a spectrum, circularly shift it by 3 bins using time-domain modulation.
3. Show that a real even sequence has an approximately real DFT.
4. Given only bins 0 through $N/2$, rebuild a full real-signal spectrum.
5. Compare unpadded and padded FFT convolution of lengths 5 and 4.
6. Multiply two polynomials whose coefficients are supplied constant-term first.
7. Repeat the polynomial problem when coefficients are supplied highest-degree first.
8. Detect a circular shift when the observed signal also has additive small noise.
9. Find the shift of each column instead of each row.
10. Build a high-pass mask by zeroing DC and nearby bins.
11. Apply a circular shift to an image using the DFT phase property rather than `np.roll`.
12. Verify Parseval for a complex-valued sequence.
13. Verify the conjugation property for a complex-valued sequence.
14. Use `ArbitraryLengthFFT` on a prime-length sequence and compare with `DFTAnalyzer`.
15. Explain why padding a true circular signal changes the modeled period.
16. Change big-integer limbs from base 10^4 to base 10^2 and update reconstruction.
17. Given a broken big-integer result, determine whether the bug is FFT precision, padding, or carry propagation.
18. Compare direct 2-D convolution and spectral convolution on a tiny 8×8 image.
19. For a motion-blur kernel, explain why the image operation is still the same convolution theorem.
20. Compute the DFT of $[1, 0, -1, 0]$ and identify the active bins.

21 Final formula sheet

Transforms

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+j2\pi kn/N}$$

$$X(e^{j\omega}) = \sum_n x[n] e^{-j\omega n}$$

$$x[n] = \frac{1}{2\pi} \int_{2\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

$$X[k] = X(e^{j\omega}) \Big|_{\omega=2\pi k/N}$$

Properties

$$ax[n] + by[n] \longleftrightarrow aX[k] + bY[k]$$

$$x[((n-m))_N] \longleftrightarrow e^{-j2\pi km/N} X[k]$$

$$e^{+j2\pi k_0 n/N} x[n] \longleftrightarrow X[((k-k_0))_N]$$

$$x[((-n))_N] \longleftrightarrow X[((-k))_N]$$

$$x^*[n] \longleftrightarrow X^*[((-k))_N]$$

$$x[n] \in \mathbb{R} \Rightarrow X[N-k] = X^*[k]$$

$$\sum_n |x[n]|^2 = \frac{1}{N} \sum_k |X[k]|^2$$

$$x \circledast h \longleftrightarrow XH$$

$$xh \longleftrightarrow \frac{1}{N} (X \circledast H)$$

$$N \geq L + M - 1 \Rightarrow \text{zero-padded circular convolution} = \text{linear convolution}$$

Correlation

Using the convention in this guide:

$$r_{xy}[m] = \text{IDFT}\{X^*[k]Y[k]\}.$$

If $y = \text{np.roll}(x, s)$, then the correlation peak occurs at s , and:

$$x = \text{np.roll}(y, -s).$$

FFT DIT butterfly

$$X[k] = G[k] + W_N^k H[k]$$

$$X[k + N/2] = G[k] - W_N^k H[k]$$

$$W_N = e^{-j2\pi/N}.$$

Sizes

$$\text{1-D linear convolution length} = L + M - 1.$$

$$\text{2-D full convolution size} = (h + k_h - 1) \times (w + k_w - 1).$$

Bins

$$\omega_k = \frac{2\pi k}{N}, \quad \Delta f = \frac{f_s}{N}.$$

$$f_k = \begin{cases} kf_s/N, & 0 \leq k \leq N/2, \\ (k - N)f_s/N, & N/2 < k < N. \end{cases}$$

22 30-minute online workflow

1. **First 2 minutes:** identify the property and output representation.
2. **Next 3 minutes:** write the mathematical formula and required length N .
3. **Next 5 minutes:** copy the closest working assignment function or copy block.
4. **Next 10 minutes:** adapt only the data preparation and spectral operation.
5. **Next 5 minutes:** verify with a tiny hand-checkable example.
6. **Final 5 minutes:** test edge cases, print error, check output order/type, and remove debugging noise if submission format requires it.

Exam Tip

When the question is convolution-like, always write `needed = len(a)+len(b)-1` before doing anything else. When the question is a property verification, always produce both `left` and `right`. When the question is a shift detector, create one synthetic `np.roll` test before trusting the sign.

23 Source-grounded closing summary

The supplied DFT-properties lecture can be summarized by its own theme: on N points, everything wraps. Circular shifts rotate the sample clock, modulation rotates the frequency clock, real signals force conjugate symmetry, and multiplying DFTs produces circular convolution. The same lecture then shows the engineering escape hatch: zero-pad until the linear convolution fits.

The FFT lecture does not change any of those DFT properties. It changes the cost of obtaining the DFT from $O(N^2)$ to $O(N \log N)$ by reusing smaller DFTs through radix-2 butterflies.

The three previous online questions are applications of those two facts:

- integer multiplication = convolution + carry;
- weighted polynomial multiplication = modified coefficients + convolution;
- row alignment = circular correlation + inverse shift.

The current assignment gives you reusable code for the same computational grammar. That is why mastering the property-to-code translation is more useful than memorizing one previous answer.

End-of-guide checklist

Before the online, make sure you can do each of these without help:

- explain DFT/IDFT signs and the $1/N$;
- explain why FFT and DFT give the same result;
- explain bit reversal and one DIT butterfly;
- choose between DFT, radix-2 FFT, and arbitrary-length FFT;
- verify every row of the supplied DFT property table;
- distinguish circular and linear convolution;
- calculate the correct padding length;
- convert polynomial/integer multiplication to convolution;
- perform carry propagation;
- compute circular cross-correlation and interpret its peak;

- perform a 2-D transform by rows then columns;
- use `np.allclose`, `np.roll`, `np.argmax`, and complex exponentials confidently.